

ACTE TECHNOLOGIES

Professional Training & Certification Institute

240

CORE JAVA INTERVIEW QUESTIONS & ANSWERS

Complete Reference for Java Developer Interviews

240 Q&A
QUESTIONS

Core Java
TECHNOLOGY

ACTE Technologies
BY

TOPICS COVERED

OOP Concepts • Inheritance • Polymorphism • Encapsulation • Abstraction
Exception Handling (try/catch/finally/throw/throws) • Checked vs Unchecked
Multithreading • Synchronization • Thread Lifecycle • wait/notify/notifyAll
Inner & Nested Classes • Interfaces • Abstract Classes • Access Modifiers
Collections Framework • ArrayList/LinkedList/HashSet/TreeSet/HashMap
Serialization • Generics • Enums • Static Imports • Varargs • Coding Standards

SECTION 1: CORE JAVA FUNDAMENTALS (Q1 – Q25)

Q1 What are static blocks and static initializers in Java?

Static blocks are used to initialize static fields. They execute exactly once when the class is loaded by the JVM — before any constructor runs.

Q2 How to call one constructor from another constructor?

Use **this()** within the same class. The correct overload is chosen based on parameters. Restrictions: **this()** must be the first statement; cannot use two **this()** calls.

Q3 What is method overriding in Java?

When a subclass defines a method with the **same name, parameters and return type** as a superclass method, it overrides it. Used when you want the same method to behave differently in subclass. At runtime, the subclass version is called.

Q4 What is the super keyword in Java?

super accesses superclass variables, methods, and constructors hidden by overriding. Two forms: calling superclass constructor **super()** (must be first statement), or accessing superclass members **super.variable**.

Q5 Difference between method overloading and method overriding?

Aspect	Method Overloading	Method Overriding
Class scope	Same class	Superclass & Subclass
Inheritance	Not required	Required
Return type	Can be different	Must be same (or covariant)
Parameters	Must be different	Must be same
Polymorphism	Static (compile-time)	Dynamic (runtime)

Q6 Difference between abstract class and interface?

Aspect	Interface	Abstract Class
Methods	Only abstract (default/static from Java 8)	Abstract, concrete, or both

Access	Methods must be public	Any except private
Variables	public, static, final	Any access specifier
Multiple inh.	Supported	Not supported
Keyword	implements	extends

Q7 Why is Java platform independent?

javac compiles Java source into **bytecode** (.class files). The JVM — available on all major platforms — interprets this bytecode into native machine code. The same bytecode runs on Windows, Linux, and Mac without recompilation.

Q8 What is method overloading?

Two or more methods in the same class with the **same name but different arguments** (type, number, or order). Achieves static (compile-time) polymorphism. Return type alone cannot distinguish overloads.

Q9 Difference between C++ and Java?

Feature	Java	C++
Platform	Platform independent (bytecode)	Platform dependent
Pointers	No pointers	Supports pointers
Op. Overloading	Not supported	Supported
Garbage Collection	Automatic GC	Manual (no GC)
Multithreading	Built-in	Not built-in
Global vars	Not allowed	Allowed

Q10 What is JIT compiler?

Just-In-Time compiler is part of the JVM that converts bytecode into native machine code at runtime — as needed — improving performance by caching frequently called code.

Q11 What is bytecode?

javac generates a .class file containing **bytecode** — machine-independent instructions understood only by the JVM. JVM interprets or JIT-compiler this into native code.

Q12 Difference between this() and super()?

this() calls another constructor in the same class. **super()** calls the superclass constructor. Both must be the very first statement in a constructor body.

Q13 What is a class?

A class is the fundamental unit of OOP — a blueprint or template defining variables and methods. Every Java program needs at least one class. Saved as a .java file with the same name as the class.

Q14 What is an object?

An object is an **instance of a class** with state (attribute values) and behaviour (methods). Created using the **new** keyword.

```
FirstClass f = new FirstClass(); // f is a reference to a FirstClass object
```

Q15 What is a method in Java?

A method contains executable code applicable to objects. It has a name, parameters, return type, and body.

```
public float add(int a, int b, int c) { ... }
```

Q16 What is encapsulation?

Encapsulation wraps data and behaviour into a single class and protects it via access control. Java provides four access modifiers: public, private, protected, default.

★ *Example: A car driver only needs start/stop — internal components are hidden.*

Q17 Why is main() public, static and void?

- **public** — accessible outside the class so JVM can invoke it.
- **static** — JVM calls main() without creating an object.
- **void** — main() does not return any value to the JVM.

Q19 What is a constructor?

A constructor initializes objects when created. No return type; same name as class. Types: **Default** (no params, compiler-generated if none defined) and **Parameterized** (accepts params).

Q20 Difference between length and length()?

	length	length()
Type	Instance variable (array)	Method (String class)
Returns	Number of array elements	Number of characters in string
Example	days.length	str.length()

Q24 What are constants?

Constants are fixed values that cannot change during execution. Created using the **final** keyword.

```
final int NUMBER = 10;
final String APP_NAME = "ACTE Java Guide";
```

SECTION 2: JAVA CODING STANDARDS (Q26 – Q30)

Q#	Category	Rules & Examples
Q26	Classes	Uppercase start, nouns, PascalCase. Ex: Employee, EmployeeDetails, ArrayList
Q27	Interfaces	Uppercase start, adjectives. Ex: Runnable, Serializable, Cloneable
Q28	Methods	Lowercase start, verbs, camelCase. Ex: toString(), getCarName(), isEmpty()
Q29	Variables	Lowercase start, nouns, camelCase, short & meaningful. Ex: empName, salary
Q30	Constants	ALL_UPPERCASE with underscores. Ex: MAX_VALUE, MIN_PRIORITY, PI

SECTION 3: OOP CONCEPTS & ADVANCED FEATURES (Q31 – Q51)

Q31 Difference between overriding and overloading?

Feature	Overriding	Overloading
Method names	Must be same	Must be same
Arguments	Must be same	Must differ
Return type	Same or covariant	Can be different
Location	Subclass only	Same class or subclass
Resolution	Runtime (dynamic)	Compile-time (static)
Also called	Dynamic polymorphism	Static polymorphism

Q32 What is 'IS-A' relationship?

IS-A = **Inheritance** via the extends keyword. Promotes reuse. Example: Car IS-A Vehicle; MotorCycle IS-A Vehicle.

Q33 What is 'HAS-A' relationship?

HAS-A = **Composition or Aggregation** using the new keyword. No special keyword. Example: Car HAS-A Engine.

Q35 Explain instanceof operator?

instanceof tests whether an object is of a particular type. Returns true/false. Compile-time check verifies compatibility; returns false if reference is null.

```
Integer a = new Integer(5);
System.out.println(a instanceof Integer); // true
```

Q45 Access Modifiers — Full Scope Comparison Table

Modifier	Same Class	Same Pkg Sub	Same Pkg Non-Sub	Diff Pkg Sub	Diff Pkg Non-Sub
public	YES	YES	YES	YES	YES
protected	YES	YES	YES	YES	NO
default	YES	YES	YES	NO	NO
private	YES	NO	NO	NO	NO

★ Top-level class: only public and default are allowed.

Q48 What is the final modifier?

Applied to	Effect
final class	Cannot be subclassed (e.g., String)
final method	Cannot be overridden in subclasses
final variable	Acts as constant — cannot be reassigned after initialization

Q49 Explain abstract classes?

An abstract class is declared with the **abstract** keyword when you cannot provide full implementation. Features: cannot be instantiated; can contain abstract and/or concrete methods; subclasses must override all abstract methods.

Q51 What are abstract methods?

Abstract methods have no body — declared with **abstract** and ending with a semicolon. The implementing subclass must provide the body.

```
public abstract void getDetails();
```


SECTION 4: EXCEPTION HANDLING (Q52 – Q81)

Q52 What is an exception in Java?

An exception is an **object representing an abnormal situation** at runtime. Can be created by JVM or application code. All exception classes are in **java.lang**.

Q54 What is Exception handling?

A mechanism to handle abnormal situations so the program does not terminate abruptly. Separates normal code from error-handling code, allowing normal execution to continue after the error.

Q57 Ways to handle exceptions:

- 1. Using **try-catch** block.
- 2. Declaring the method with a **throws** clause to delegate to the caller.

Q58 Five keywords for exception handling:

Keyword	Purpose
try	Wraps code that may throw an exception
catch	Catches and handles exception thrown by try block
finally	Always executes — used for resource cleanup
throw	Explicitly throws an exception object
throws	Declares that a method may throw checked exceptions

Q61 Can we have multiple catch blocks?

Yes. Each catch handles a different exception type. JVM executes the first matching catch. **Order must be child-to-parent** (most specific first). From Java 7, multiple types in one catch: **catch (ExA | ExB e)**.

Q62 Importance of finally block?

finally always executes (after try or catch) — perfect for resource cleanup. Exception: **System.exit(0)** in try prevents finally from running.

Q68 Checked vs Unchecked Exceptions?

Aspect	Checked	Unchecked
Hierarchy	Subclasses of Throwable (excl. RuntimeException and Error)	Subclasses of RuntimeException
Compile check	Must handle (try-catch or throws)	Compiler does not enforce
Examples	IOException, SQLException, ClassNotFoundException, NullPointerException	NullPointerException, ArithmeticException, ClassCastException

Q70 Explain throw keyword?

throw explicitly throws an exception. Execution stops after throw; remaining statements are unreachable. Used mainly for user-defined exceptions.

```
throw new IllegalArgumentException("Value must be positive");
```

Q72 Explain throws keyword?

throws in a method signature delegates checked exception handling to the caller. Not required for unchecked exceptions.

```
public void readFile(String path) throws IOException { ... }
```

Q76 User-defined exceptions?

Custom exceptions provide domain-specific error messages. Extend **RuntimeException** for unchecked (recommended) or **Exception** for checked.

```
public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String msg) { super(msg); }
}
```

Q79 Throwable class and its methods?

Method	Description
printStackTrace()	Prints exception name, description, and full stack trace
getMessage()	Returns only the description of the exception
toString()	Returns name and description of the exception

SECTION 5: MULTITHREADING & SYNCHRONIZATION (Q83 – Q138)

Q85 Difference between Process and Thread?

Aspect	Process	Thread
Definition	Program in execution	Path of execution within a process
Weight	Heavy weight	Light weight
Memory	Separate address space	Shared address space
Communication	Expensive	Less expensive
Context switch	Costly	Low cost

Q92 Ways to create threads?

Approach	Code Pattern	Best?
Extend Thread	class MyThread extends Thread { void run(){...} }	No — prevents other inheritance
Implement Runnable	class MyTask implements Runnable { void run(){...} }	YES — allows extending other classes

Q97 Thread lifecycle (5 states):

State	Description
New	Instance created; start() not yet called.
Runnable	start() called or returned from blocked/sleeping state.
Running	Scheduler allocated CPU time; thread is executing.
Waiting/Blocked/Sleeping	Temporarily inactive: waiting for lock, notification, or sleep timer.
Dead	run() completed. Cannot be restarted.

Q106 Synchronization methods in Java?

Synchronization prevents multiple threads from accessing a shared resource simultaneously. Two ways:

- **Synchronized method** — entire method locked on object.
- **Synchronized block** — granular; locks only critical code section. Better performance.

```
public synchronized void deposit(int amount) { } // method
synchronized (accountObj) { balance += amount; } // block
```

Q120 Methods to prevent/control thread execution:

Method	Action	Releases Lock?
yield()	Moves thread to runnable; gives chance to equal-priority threads	No
sleep(ms)	Pauses current thread for specified time	No
join()	Caller waits for joined thread to finish	No
wait()	Releases lock; waits for notify()	YES
notify()	Wakes one waiting thread on the object	YES
notifyAll()	Wakes all waiting threads; highest priority runs first	YES

Q129 Explain wait(), notify(), notifyAll()?

These three methods of **Object class** enable inter-thread communication. They must always be called inside a **synchronized** context — otherwise `IllegalMonitorStateException` is thrown.

★ They are in *Object class (not Thread)* because they operate on the shared object lock, not on threads.

Q136 Daemon threads?

Background service threads (e.g., Garbage Collector). Use **setDaemon(true)** before `start()` to make a thread daemon. Main thread is always non-daemon and cannot be changed.

SECTION 6: NESTED & INNER CLASSES (Q139 – Q165)

Q139 Types of nested classes?

Type	Static?	Description
Static Nested Class	Yes	Can instantiate without outer instance; no access to outer instance members
Member Inner Class	No	Non-static; has access to outer class private members
Local Inner Class	No	Defined inside a method; exists only during method execution
Anonymous Inner Class	No	No class name; declared and instantiated simultaneously using new

Q141 Why use nested classes?

- **Grouping** — classes not reusable elsewhere can be inner classes.
- **Encapsulation** — inner classes can access outer class private members.
- **Readability** — avoids creating unrelated top-level classes.
- **Implementation hiding** — hides implementation details.

Q146 Anonymous inner classes?

Declared and instantiated simultaneously with no name. Primary use: one-time interface implementations.

```
Runnable r = new Runnable() {
    @Override public void run() { System.out.println("Running"); }
};
```

Q151 How to implement encapsulation?

- Declare all instance variables **private**.
- Provide **public getter and setter** methods to access/modify them.

Q163 Wrapper classes table?

Primitive	Wrapper
boolean	Boolean
int	Integer
float	Float

char	Character
byte	Byte
long	Long
short	Short

★ From Java 1.5, autoboxing converts primitives to wrapper objects automatically.

SECTION 7: INTERFACES, ENUMS & OOP PRINCIPLES (Q179 – Q197)

Q179 Define interface in Java?

An interface is a 100% abstract class — a contract that defines what a class should do. Implemented using the **implements** keyword. From Java 8, interfaces can have default and static methods.

Q181 Features of interfaces?

Feature	Detail
Methods	Implicitly public and abstract (default/static from Java 8)
Variables	Implicitly public, static, final
Instantiation	Cannot be instantiated directly
Multiple inheritance	A class can implement multiple interfaces
Interface extension	An interface can extend multiple interfaces

Q182 Explain enumeration (enum)?

Enum (Java 5) is a set of named constants — each is public, static, and final. Restrictions: cannot extend; cannot be instantiated with new; fields/methods must follow enum constants.

```
public enum Days { SUN, MON, TUE, WED, THU, FRI, SAT; }
```

Q193 Three core OOP principles:

Principle	Definition
Encapsulation	Wrapping data + methods in a class; controlling access via modifiers.
Inheritance	Subclass acquires members of superclass using extends. Promotes reusability.
Polymorphism	One interface, many forms. Runtime type determines which method is called.

SECTION 8: COLLECTIONS FRAMEWORK (Q198 – Q230)

Q200 Difference between collection, Collection, Collections?

Term	Meaning
collection (lowercase)	Concept — a group of objects.
Collection (interface)	Root interface: add(), remove(), iterator(), size()...
Collections (class)	Utility class with static methods: sort(), synchronizedList()...

Q207 Array vs ArrayList?

Aspect	Array	ArrayList
Size	Fixed at creation	Dynamic (auto-grows)
Types	Primitives or objects	Objects only (autoboxed)
Speed	Faster	Slightly slower
Insert/Remove	Manual logic	Built-in add()/remove()
Nature	Static	Dynamic (backed by array)

Q209 ArrayList vs Vector?

Aspect	ArrayList	Vector
Synchronization	Not synchronized	Synchronized
Performance	Faster	Slower
Introduced	Java 2.0	Java 1.0 (legacy)

Q215 Set implementations comparison?

Class	Order	Nulls	Sorted	Use When
HashSet	Unordered	1 null	No	Fastest; no order needed
TreeSet	Natural sort	No null	Yes	Need sorted elements
LinkedHashSet	Insertion order	1 null	No	Need insertion-order iteration

Q223 HashMap vs Hashtable?

Aspect	HashMap	Hashtable
Synchronization	Not synchronized	Synchronized
Nulls	1 null key + any null values	No nulls allowed
Performance	Faster	Slower
Introduced	Java 2.0	Java 1.0 (legacy)

Q224

ArrayList vs LinkedList?

Aspect	ArrayList	LinkedList
Access	RandomAccess (fast indexed get)	Sequential access only
Search	Fast (random)	Slow (sequential)
Add/Remove (middle)	Slow (array shift)	Fast (just update node refs)
Memory	Less (array)	More (node pointers)

Q225

Comparator vs Comparable?

Aspect	Comparable	Comparator
Package	java.lang	java.util
Purpose	Natural ordering (self)	Custom ordering (external)
Method	compareTo(T o)	compare(T o1, T o2)
Sort call	Collections.sort(list)	Collections.sort(list, comparator)

Q229

Fail-fast vs Fail-safe iterators?

Type	Throws ConcurrentModificationException?	Examples
Fail-fast	YES — if collection modified during iteration	ArrayList, HashSet, HashMap iterators
Fail-safe	NO — works on a copy of the collection	CopyOnWriteArrayList, ConcurrentHashMap

SECTION 9: SERIALIZATION (Q231 – Q240)

Q231 What is serialization?

Serialization converts an object into a **byte stream** for network transmission or file storage. Deserialization recreates the object from the byte stream.

Q232 Main purposes of serialization:

Purpose	Description
Persistence	Store object state in a file/database for later use
Communication	Transmit objects over a network via RPC
Copying	Create duplicates from byte arrays
Distribution	Share objects across different JVMs

Q234 Serializable interface?

java.io.Serializable is a **marker interface** with no methods. Implementing it signals the JVM that instances can be serialized.

Q236 What is serialVersionUID?

A 64-bit long identifier used to verify sender/receiver class compatibility. Mismatch throws **InvalidClassException**. Define explicitly to avoid JVM computation overhead.

```
private static final long serialVersionUID = -5885568094444284875L;
```

Q238 Can static variables be serialized?

No. Static variables belong to the class, not to object instances. Serialization saves object state only. Exception: serialVersionUID itself is serialized with the object.

Q240 How to prevent specific fields from being serialized?

Declare them as **transient**. During deserialization, transient fields get default values (0 / null).

```
public class Car implements Serializable {
    transient int carNumber; // will NOT be serialized
}
```

ACTE TECHNOLOGIES
www.acte.in

QUICK REFERENCE — ALL 240 QUESTIONS AT A GLANCE

Range	Topics Covered
Q1–Q11	Static blocks, Constructor chaining, Overriding, super, Overloading vs Overriding, Interface vs Abstract, Platform in
Q12–Q25	this() vs super(), Class, Object, Method, Encapsulation, main() details, Constructor, length vs length(), ASCII, Unico
Q26–Q30	Java Coding Standards — Classes, Interfaces, Methods, Variables, Constants
Q31–Q51	Overriding vs Overloading table, IS-A/HAS-A relationships, instanceof, Access modifiers, final modifier, Abstract cla
Q52–Q81	Exception types & hierarchy, try/catch/finally/throw/throws, Checked vs Unchecked, Default handling, User-defined
Q83–Q115	Process vs Thread, Multitasking types, Thread lifecycle, Thread creation (Runnable vs Thread), Thread scheduler,
Q116–Q138	Thread priorities, yield()/join()/sleep(), wait()/notify()/notifyAll(), Synchronized methods/blocks, Class-level lock, Thre
Q139–Q150	Nested classes — Static nested, Method local inner, Anonymous inner, Member inner — features & instantiation
Q151–Q188	Encapsulation how-to, Reference variables, Constructors, Static methods, Wrapper classes, Type conversion, Pack
Q189–Q197	Procedural vs OOP, OOP benefits, Inheritance, Polymorphism, Encapsulation deep dive, Covariant return
Q198–Q230	Collections framework, List/Set/Queue, ArrayList/Vector/LinkedList, Iterator/ListIterator, HashSet/TreeSet/LinkedHa
Q231–Q240	Serialization purpose, Serializable interface, serialVersionUID, transient keyword, static variable serialization, refere